

ASSIGNMENT 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

df=pd.read_csv("day.csv")
print(df.head())

print()
print("printing info of the dataset:")
print()

print(df.info)
print()

print(df.shape)
print()

print(df.isnull().sum())    #to check if there are any null values in the dataset
print()

print("converting 'dteday' column to DATETIME")
df['dteday']=pd.to_datetime(df['dteday'], format='%d-%m-%Y')

print(df['dteday'])
print()
```

```
PS C:\Users\DELL\Desktop\assignment1_ML> & C:/Users/DELL/anaconda3/python.exe  
c:/Users/DELL/Desktop/assignment1_ML/assignment.py
```

[illegible]

```

1    2 02-01-2018    1 0 1    0 2    1    2 14.902598 17.68695 69.6087 16.652113
131    670 801

2    3 03-01-2018    1 0 1    0 3    1    1 8.050924 9.47025 43.7273 16.636703
120    1229 1349

3    4 04-01-2018    1 0 1    0 4    1    1 8.200000 10.60610 59.0435 10.739832
108    1454 1562

4    5 05-01-2018    1 0 1    0 5    1    1 9.305237 11.46350 43.6957 12.522300
82    1518 1600

```

printing info of the dataset:

```

<bound method DataFrame.info of      instant    dteday season yr mnth holiday weekday workingday
weathersit    temp    atemp    hum windspeed casual registered cnt

```

```

0    1 01-01-2018    1 0 1    0 1    1    2 14.110847 18.18125 80.5833 10.749882
331    654 985

1    2 02-01-2018    1 0 1    0 2    1    2 14.902598 17.68695 69.6087 16.652113
131    670 801

2    3 03-01-2018    1 0 1    0 3    1    1 8.050924 9.47025 43.7273 16.636703
120    1229 1349

3    4 04-01-2018    1 0 1    0 4    1    1 8.200000 10.60610 59.0435 10.739832
108    1454 1562

4    5 05-01-2018    1 0 1    0 5    1    1 9.305237 11.46350 43.6957 12.522300
82    1518 1600

..    ..    ..    .. ..    ..    ..    ..    ..    ..    ..    ..    ..    ..    ..

725    726 27-12-2019    1 1 12    0 5    1    2 10.420847 11.33210 65.2917
23.458911    247    1867 2114

726    727 28-12-2019    1 1 12    0 6    0    2 10.386653 12.75230 59.0000
10.416557    644    2451 3095

727    728 29-12-2019    1 1 12    0 0    0    2 10.386653 12.12000 75.2917
8.333661    159    1182 1341

```

```
728  729 30-12-2019  1  1  12  0  1  1  1  10.489153  11.58500  48.3333
23.500518  364  1432  1796
```

```
729  730 31-12-2019  1  1  12  0  2  1  2  8.849153  11.17435  57.7500
10.374682  439  2290  2729
```

```
[730 rows x 16 columns]>
```

```
(730, 16)
```

```
instant    0
```

```
dteday     0
```

```
season     0
```

```
yr         0
```

```
mnth       0
```

```
holiday     0
```

```
weekday    0
```

```
workingday  0
```

```
weathersit  0
```

```
temp        0
```

```
atemp       0
```

```
hum         0
```

```
windspeed  0
```

```
casual      0
```

```
registered  0
```

```
cnt         0
```

```
dtype: int64
```

converting 'dteday' column to DATETIME

```
0    2018-01-01
1    2018-01-02
2    2018-01-03
3    2018-01-04
4    2018-01-05
...
725  2019-12-27
726  2019-12-28
727  2019-12-29
728  2019-12-30
729  2019-12-31
```

Name: dteday, Length: 730, dtype: datetime64[ns]

```
df['dteday']=pd.to_datetime(df['dteday'], format='%d-%m-%Y')

print(df['dteday'])
print()

cat_cols=['season','yr','mnth','holiday','weekday','workingday','weathersit']
for col in cat_cols:
    df[col]=df[col].astype('category')
    #print(df[col])

X=df.drop(columns=['instant','dteday','casual','registered','cnt'])
Y=df['cnt']

print(X)
print(Y)

#X=pd.get_dummies(X,drop_first=True)
```

```

X_train, X_test, Y_train, Y_test= train_test_split(X, Y, test_size=0.2,
random_state=42)

X_train_sm= sm.add_constant(X_train)
ols=sm.OLS(Y_train, X_train_sm).fit()
print(ols.summary())

print()

```

	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed
0	1	0	1	0	1	1	2	14.110847	18.18125	80.5833	10.749882
1	1	0	1	0	2	1	2	14.902598	17.68695	69.6087	16.652113
2	1	0	1	0	3	1	1	8.050924	9.47025	43.7273	16.636703
3	1	0	1	0	4	1	1	8.200000	10.60610	59.0435	10.739832
4	1	0	1	0	5	1	1	9.305237	11.46350	43.6957	12.522300
..	...	...	...	...	...	...	...	...	...	...	...
725	1	1	12	0	5	1	2	10.420847	11.33210	65.2917	23.458911
726	1	1	12	0	6	0	2	10.386653	12.75230	59.0000	10.416557
727	1	1	12	0	0	0	2	10.386653	12.12000	75.2917	8.333661
728	1	1	12	0	1	1	1	10.489153	11.58500	48.3333	23.500518
729	1	1	12	0	2	1	2	8.849153	11.17435	57.7500	10.374682

[730 rows x 11 columns]

```

0    985
1    801
2   1349

```

3 1562
4 1600
...
725 2114
726 3095
727 1341
728 1796
729 2729

Name: cnt, Length: 730, dtype: int64

OLS Regression Results

=====			
Dep. Variable:	cnt	R-squared:	0.787
Model:	OLS	Adj. R-squared:	0.783
Method:	Least Squares	F-statistic:	191.7
Date:	Tue, 21 Oct 2025	Prob (F-statistic):	1.04e-183
Time:	13:55:11	Log-Likelihood:	-4802.7
No. Observations:	584	AIC:	9629.
Df Residuals:	572	BIC:	9682.
Df Model:	11		
Covariance Type:	nonrobust		
=====			

	coef	std err	t	P> t	[0.025	0.975]

const	1605.2576	292.631	5.486	0.000	1030.496	2180.019
season	535.5726	67.466	7.938	0.000	403.061	668.084

yr	1997.4375	76.440	26.131	0.000	1847.300	2147.575
mnth	-50.1648	21.205	-2.366	0.018	-91.815	-8.515
holiday	-802.3747	223.231	-3.594	0.000	-1240.826	-363.923
weekday	20.0210	19.064	1.050	0.294	-17.423	57.465
workingday	-198.5230	85.635	-2.318	0.021	-366.721	-30.325
weathersit	-598.0214	90.537	-6.605	0.000	-775.847	-420.196
temp	-31.2016	65.048	-0.480	0.632	-158.964	96.561
atemp	148.0836	60.696	2.440	0.015	28.869	267.298
hum	-9.8799	3.668	-2.694	0.007	-17.084	-2.676
windspeed	-36.7692	8.333	-4.413	0.000	-53.135	-20.403

```
=====
```

Omnibus:	75.983	Durbin-Watson:	2.010
Prob(Omnibus):	0.000	Jarque-Bera (JB):	151.017
Skew:	-0.755	Prob(JB):	1.61e-33
Kurtosis:	4.982	Cond. No.	586.

```
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
def select_significant_features(X, Y, significance_level=0.05):
    X_const = X.copy()
    while True:
        X_sm=sm.add_constant(X_const)
        ols=sm.OLS(Y, X_sm).fit()
        p_values=ols.pvalues.drop('const')
        max_p = p_values.max()
        if max_p > significance_level:
            remove_feature = p_values.idxmax()
```

```

        X_const.drop(remove_feature, axis=1, inplace=True)
    else:
        break
    return ols, X_const
ols_final, X_train_final=select_significant_features(X_train,Y_train)
print(ols_final.summary())

print()
print("Printing the significant variables :")
significant_vars = ols_final.pvalues[ols_final.pvalues < 0.05]
print("Significant predictors of bike demand:", significant_vars.index.tolist())
print()

```

OLS Regression Results

```

=====
Dep. Variable:          cnt  R-squared:          0.786
Model:                  OLS  Adj. R-squared:      0.783
Method:                 Least Squares  F-statistic:      234.5
Date:                  Tue, 21 Oct 2025  Prob (F-statistic):  8.40e-186
Time:                  13:55:12  Log-Likelihood:    -4803.4
No. Observations:      584  AIC:                  9627.
Df Residuals:          574  BIC:                  9671.
Df Model:              9
Covariance Type:       nonrobust

```

```

=====
               coef  std err          t  P>|t|  [0.025  0.975]
-----
const    1721.3724   261.276    6.588   0.000   1208.199   2234.545
season     538.2509    67.335    7.994   0.000    405.998    670.504
yr      1995.3194    76.270   26.161   0.000   1845.518   2145.121

```


mnth	-50.6076	21.189	-2.388	0.017	-92.224	-8.991
holiday	-815.0000	222.438	-3.664	0.000	-1251.892	-378.108
workingday	-201.9969	85.515	-2.362	0.019	-369.957	-34.037
weathersit	-598.4909	89.643	-6.676	0.000	-774.559	-422.423
atemp	119.3673	5.155	23.157	0.000	109.243	129.492
hum	-9.7288	3.631	-2.679	0.008	-16.861	-2.596
windspeed	-38.0672	8.086	-4.708	0.000	-53.948	-22.186

```
=====
```

Omnibus:	75.839	Durbin-Watson:	2.021
Prob(Omnibus):	0.000	Jarque-Bera (JB):	149.393
Skew:	-0.757	Prob(JB):	3.63e-33
Kurtosis:	4.962	Cond. No.	501.

```
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Printing the significant variables :

Significant predictors of bike demand: ['const', 'season', 'yr', 'mnth', 'holiday', 'workingday', 'weathersit', 'atemp', 'hum', 'windspeed']

Variance Infation Factor (VIF):

	Feature	VIF
0	season	24.221420
1	yr	1.996659

2 mnth 17.405227

3 holiday 1.116429

4 workingday 3.449126

5 weathersit 12.911730

6 atemp 10.631320

=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Printing the significant variables :

Significant predictors of bike demand: ['const', 'season', 'yr', 'mnth', 'holiday', 'workingday', 'weathersit', 'atemp', 'hum', 'windspeed']

```
vif_df=pd.DataFrame()
vif_df["Feature"]=X_train_final.columns
vif_df["VIF"]= [variance_inflation_factor(X_train_final.values, i)
for i in range(X_train_final.shape[1])]
print("\n Variance Infation Factor (VIF): \n", vif_df)
print()

X_test_final = X_test[X_train_final.columns]
linear=LinearRegression()
linear.fit(X_train_final, Y_train)
Y_pred=linear.predict(X_test_final)

r2= r2_score(Y_test, Y_pred)
adj_r2=1-(1-r2)*(len(Y_test)-1)/(len(Y_test)-X_test_final.shape[1]-1)
rmse= np.sqrt(mean_squared_error(Y_test,Y_pred))

print()
print("Model evaluation metrics :")
print()
print(f"r2 score : {r2:.4f}")
print()
```

```
print(f"Adjusted r2 : {adj_r2:.4f}")
print()
print(f"rmse : {rmse:.4f}")
```

Variance Infation Factor (VIF):

	Feature	VIF
0	season	24.221420
1	yr	1.996659
2	mnth	17.405227
3	holiday	1.116429
4	workingday	3.449126
5	weathersit	12.911730
6	atemp	10.631320

Variance Infation Factor (VIF):

	Feature	VIF
0	season	24.221420
1	yr	1.996659
2	mnth	17.405227
3	holiday	1.116429
4	workingday	3.449126
5	weathersit	12.911730
6	atemp	10.631320

	Feature	VIF
--	---------	-----

```
0 season 24.221420
1 yr 1.996659
2 mnth 17.405227
3 holiday 1.116429
4 workingday 3.449126
5 weathersit 12.911730
6 atemp 10.631320
2 mnth 17.405227
3 holiday 1.116429
4 workingday 3.449126
5 weathersit 12.911730
6 atemp 10.631320
3 holiday 1.116429
4 workingday 3.449126
5 weathersit 12.911730
6 atemp 10.631320
5 weathersit 12.911730
6 atemp 10.631320
6 atemp 10.631320
7 hum 27.273900
8 windspeed 5.470190
8 windspeed 5.470190
```

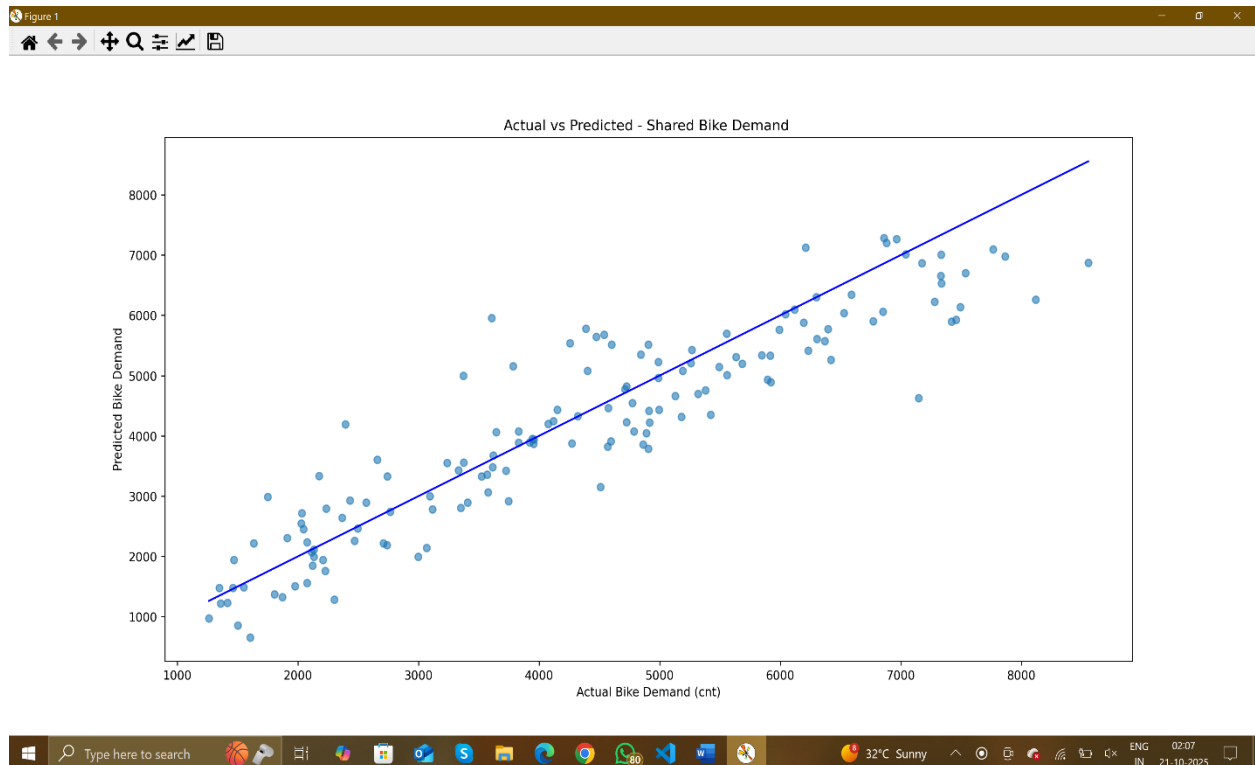
Model evaluation metrics :

r2 score : 0.8381

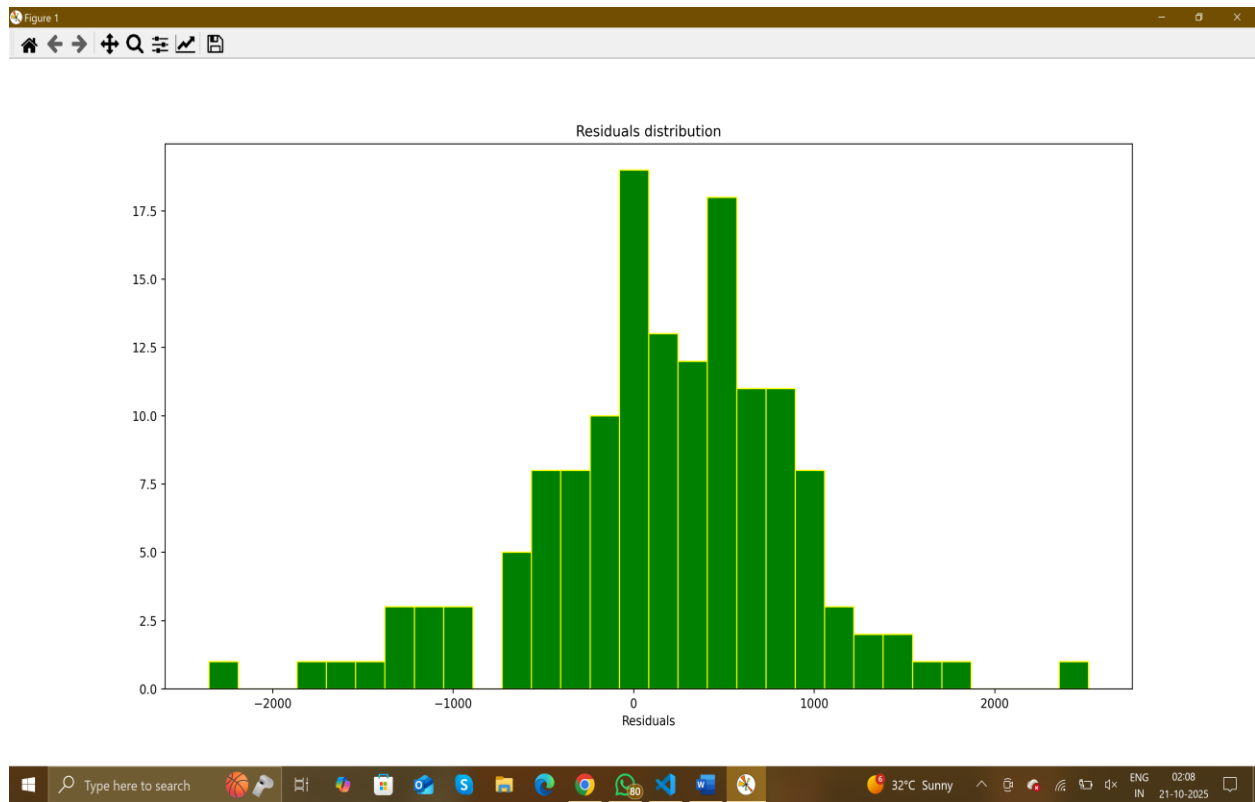
Adjusted r2 : 0.8274

rmse : 744.4954

```
plt.figure(figsize=(8,6))
plt.scatter(Y_test,Y_pred, alpha=0.6)
plt.xlabel("Actual Bike Demand (cnt)")
plt.ylabel("Predicted Bike Demand")
plt.title("Actual vs Predicted - Shared Bike Demand ")
plt.plot([Y_test.min(), Y_test.max()], [Y_test.min(), Y_test.max()], color="blue")
plt.show()
```



```
plt.figure(figsize=(8,5))
residuals=Y_test-Y_pred
plt.hist(residuals, bins=30, color='green' , edgecolor='yellow')
plt.title("Residuals distribution")
plt.xlabel("Residuals")
plt.show()
```



```
print("\n Final summary:")
print(f"Total features initially : {X.shape[1]}")
print(f"Selected features : {X_test_final.shape[1]}")
print(f"Significant variables : {significant_vars.index.tolist()}")
print(f"r2= {r2:.4f}, Adjusted r2 = {adj_r2:.4f}, rmse = {rmse:.4f}")
```

Final summary:

Total features initially : 11

Selected features : 9

Significant variables : ['const', 'season', 'yr', 'mnth', 'holiday', 'workingday', 'weathersit', 'atemp', 'hum', 'windspeed']

$r^2 = 0.8381$, Adjusted $r^2 = 0.8274$, rmse = 744.4954

PS C:\Users\DELL\Desktop\assignment1_ML>